

EXPERIMENT – 6

Name: Aryan	UID: 24BAI70179
Section/Group: 24AIT NTPP 2A	Branch/Semester: CSE_AIML (CS221) / 5th
Subject Code: 24 CSP-337	Subject Name: Full Stack - II
Date of Performance:	Submitted to: Ms. Lovleen Kaur (E20454)

Experiment – 6.1

1. AIM

To implement server-side pagination and sorting for the User REST API using Spring Data Pageable, and to display paginated results in the frontend.

2. OBJECTIVE

1. To understand Pageable, PageRequest, Sort and the Page interface provided by Spring Data.
2. To expose a GET endpoint that accepts page, size and sort query parameters.
3. To return a custom PageResponse containing content, page number, page size, total elements and total pages.
4. To use @PageableDefault so that sensible defaults apply when the client omits parameters.
5. To build UI controls for next/previous page and page-size selection.

3. SOFTWARE REQUIREMENT

1. Java JDK 17 or above and Maven Wrapper.
2. Visual Studio Code with Extension Pack for Java.
3. Spring Boot 3.4.x, Spring Web, Spring Data JPA, Validation, H2.
4. Web browser and Live Server for the frontend.

4. THEORY

Pagination avoids loading an entire table into memory and over the network. Spring Data accepts an argument of type Pageable in a repository or service method. Spring MVC binds the query string ?page=0&size=5&sort=name,asc to a PageRequest automatically.

The repository method findAll(Pageable) returns Page<T>. Page carries the slice of entities plus metadata: getNumber(), getSize(), getTotalElements(), getTotalPages(), isFirst(), isLast().

A custom DTO such as `PageResponse` is useful so that the JSON seen by the frontend is stable and does not expose internal `PageImpl` fields. Sorting is expressed as `Sort.by("name").ascending()` or as the sort query parameter.

`@PageableDefault(size = 5, sort = "id")` documents the intended default and protects the server when the client sends no parameters.

5. IMPLEMENTATION/PROCEDURE

1. Start from the Experiment 5 REST project (entity, repository, service, controller).
2. Add `PageResponse` DTO with fields `content`, `page`, `size`, `totalElements`, `totalPages`.
3. Change `UserService getUsers` to accept `Pageable` and call `userRepository.findAll(pageable)`.
4. Change `UserController GET /api/users` to take `@PageableDefault Pageable pageable` and wrap the result in `PageResponse`.
5. Keep create and delete endpoints unchanged.
6. Update the frontend to call `/api/users?page=0&size=5` and to render next/previous buttons using `totalPages`.
7. Run the backend, insert several users, and verify that changing page and size changes the returned slice.

6. CODE

Controller signature:

```
@GetMapping
public ResponseEntity<PageResponse<User>> getUsers(
    @PageableDefault(size = 5, sort = "id") Pageable pageable) {
    return ResponseEntity.ok(userService.getUsers(pageable));
}
```

Service:

```
public PageResponse<User> getUsers(Pageable pageable) {
    Page<User> page = userRepository.findAll(pageable);
    return new PageResponse<>(
        page.getContent(),
        page.getNumber(),
        page.getSize(),
        page.getTotalElements(),
        page.getTotalPages()
    );
}
```

7. OUTPUT

Experiment 6.1

Pagination & Sorting REST API

Add User

UID

Name

24BAI70179

ARYAN

Add User

Pagination & Sorting

Users Per Page

Sort By

Order

3

UID

Ascending

Apply

Users

Total Users: 4

Refresh

UID	NAME	ACTION
222222	ABABA	Delete
24BAI0000	ACACACA	Delete
24BAI70179	ARYAN	Delete

Previous

Page 1 of 2

Next

User added successfully.

Figure 1: Output of Experiment 6.1

8. LEARNING OUTCOME

- Understood Spring Data pagination and the Pageable / Page abstractions.
- Exposed page, size and sort as query parameters on a REST endpoint.
- Returned a clear PageResponse DTO suitable for a frontend.
- Connected pagination metadata to next/previous controls in the UI.

Experiment – 6.2

1. AIM

To improve the performance of the User API by introducing query optimization (JOIN FETCH and native SQL), result sorting, and Spring Cache with cache eviction on write operations.

2. OBJECTIVE

1. To model a one-to-one relationship between User and Address and observe lazy loading.
2. To compare a normal JPQL query with a JOIN FETCH query that avoids the N+1 select problem.
3. To write a native SQL join that returns selected columns.
4. To enable Spring Cache (@EnableCaching, @Cacheable, @CacheEvict) with the simple in-memory cache.
5. To provide sort-by-id and sort-by-name repository methods.

3. SOFTWARE REQUIREMENT

1. Java JDK 17 or above and Maven Wrapper.
2. Spring Boot 3.4.x with Web, Data JPA, Validation, Cache and H2.
3. Visual Studio Code, browser and Live Server.

4. THEORY

When User has a lazy OneToOne Address, calling findAll() and then reading address from each user can trigger one extra SQL statement per user. This is the N+1 problem. JOIN FETCH tells JPQL to load User and Address in a single select using an SQL JOIN.

A native query uses plain SQL. It is useful when the projection is a subset of columns or when database-specific features are required. The result type is often List<Object[]> unless an interface projection is defined.

Spring Cache is an abstraction. With spring.cache.type=simple the cache lives in the JVM heap. @Cacheable(value="users", key="allUsers") stores the method return value. The next call with the same key returns the cached list and does not hit the database. @CacheEvict(allEntries = true) on create and delete keeps the cache consistent after writes.

@Transactional(readOnly = true) keeps the persistence context open for the duration of the service method so that lazy associations can be initialized before the session closes (or JOIN FETCH can be used so initialization is unnecessary).

5. IMPLEMENTATION/PROCEDURE

1. Add Address entity with city and country, mapped OneToOne to User (addresses table, user_id foreign key).
2. Extend UserRequest with city and country. Persist Address together with User (cascade ALL).

3. Add repository methods: `findUsersNormal()`, `findUsersWithAddress()` using JOIN FETCH, `findUsersNative()` as native SQL, `findAllByIdAsc()`, `findAllByNameAsc()`.
4. Add `CacheConfig` with `@EnableCaching`. Set `spring.cache.type=simple`.
5. Annotate `getUsersCached` with `@Cacheable` and create/delete with `@CacheEvict`.
6. Expose endpoints: POST `/api/users`, GET `/normal`, `/optimized`, `/cached`, `/native`, `/sort/id`, `/sort/name`, DELETE `/api/users/{id}`.
7. Build a light-blue frontend with buttons for each query type and a table of id, uid, name, city, country.
8. Run the application, insert users, click Normal Query then JOIN FETCH, and compare SQL printed by Hibernate. Click Cached Query twice and note that the second call is served from cache. Add or delete a user and confirm the cache is cleared.

6. CODE

Repository queries:

```
public interface UserRepository extends JpaRepository<User, Long> {
    @Query("SELECT u FROM User u")
    List<User> findUsersNormal();

    @Query("SELECT u FROM User u LEFT JOIN FETCH u.address")
    List<User> findUsersWithAddress();

    @Query(value = "SELECT u.id, u.uid, u.name, a.city, a.country
        FROM users u LEFT JOIN addresses a ON u.id = a.user_id",
        nativeQuery = true)
    List<Object[]> findUsersNative();

    List<User> findAllByIdAsc();
    List<User> findAllByNameAsc();
}
```

Caching on the service:

```
@CacheEvict(value = "users", allEntries = true)
public User createUser(UserRequest request) { ... }

@Cacheable(value = "users", key = "'allUsers'")
@Transactional(readOnly = true)
public List<User> getUsersCached() {
    return userRepository.findUsersWithAddress();
}

@CacheEvict(value = "users", allEntries = true)
public void deleteUser(Long id) { ... }
```

7. OUTPUT

Experiment 6.2

Caching & Query Optimization

Add User

UID

Name

City

Country

Add User

Query Optimization Tests

Normal Query

JOIN FETCH

Cached Query

Native SQL

Sort Users

Sort by ID

Sort by Name

Refresh

SORT ID completed in 10.00 ms

User List

ID	UID	NAME	CITY	COUNTRY	ACTION
1	24BAI70179	ARYAN	Chandigarh	India	Delete
2	24BAI00000	ARYAN	Noida	India	Delete
3	24BAI11111	ARYAN	London	UK	Delete

Figure 2: Output of Experiment 6.2

8. LEARNING OUTCOME

- Identified the N+1 select problem and solved it with JOIN FETCH.
- Wrote a native SQL join and consumed Object[] results in the frontend.
- Applied Spring Cache with @Cacheable and kept it correct using @CacheEvict.
- Compared query strategies using Hibernate SQL logging and UI timings.